

오늘의 작업기록 — 2026년 6월 25일

mini-ledger 증분 3 — 커스텀 예외 + try/catch

(실패에 이름을 붙이고, 그 실패를 잡아 "돈 증발 없음"을 증명한 날)

한눈에

항목	내용
목표	증분 3 — 커스텀 예외 2개 + try/catch로 ②③ 케이스 최종 증명
결과	완료 ✓ — 예외 2개 + 배선 + try/catch, 컴파일·실행·푸시
커밋	532825f "add custom exceptions and try/catch tests" (5 files, 예외 2개 신규)
푸시	9cc49e6..532825f main -> main
저장소	github.com/Taewoo-Won/mini-ledger
새 개념	상속(extends) · super(message) · checked/unchecked · try/catch · throw/catch 분리
핵심 성취	"이체 후 a 잔액 1100" → 돈 증발 없음을 프로그램이 증명 (6/21부터의 스레드 종료)
정직한 경고	오늘 직접 친 게 거의 없다. 빈칸 채우기 + heredoc 복붙으로 넘어갔다. → 그래서 안 남았다. 이 기록이 그 빛을 갠 도구다. (회고 참조)

0. 이 기록을 왜 이렇게 길게 쓰나 — 정직한 전제

오늘은 시간이 20분뿐이었고, 그래서 나(클로드)에게 의존이 과했다. 예외 클래스 3줄도, try/catch 틀도 내가 썼고, 너는 빈칸 두어 개만 채웠다. 코드를 손으로 친 게 아니라 복붙으로 넘겼다. 돌아가는 코드는 나왔지만 — 머리에 남은 건 별로 없다. 너도 그걸 안다.

그래서 이 문서는 "오늘 한 작업"의 기록이 아니다. 네가 직접 했어야 할 이해를 통째로 다시 까는 학습 문서다. 아래를 읽고, 마지막 검증 질문에 답하고, 내일 이 파일을 덮고 빈 화면에서 직접 쳐라. 그래야 오늘이 진짜로 끝난다.

읽는 법: 각 PART의 "왜"를 네 말로 다시 설명할 수 있는지 자문하면서 읽어라. 못 하면 그 PART를 다시 읽어라.

PART 1 — 왜 커스텀 예외인가 (결제의 본질)

1-1. IllegalArgumentException의 한계

지금까지 계좌가 없을 때도, 잔액이 부족할 때도 전부 IllegalArgumentException / IllegalStateException 을 던졌다. 이건 호출자에게 딱 한 마디만 한다: "뭔가 잘못됐어."

근데 결제는 실패 경로가 곧 제품이다. 실패했을 때 왜 실패했는지에 따라 완전히 다르게 대응해야 한다:

실패 이유	대응
계좌가 없음	"가입하시겠어요?" → 가입 유도
잔액이 부족	"충전하시겠어요?" → 충전 유도

IllegalArgumentException 하나로써는 이 둘을 구분할 수 없다. 호출자가 "계좌가 없어서야? 잔액이 부족해서야?"를 알려면 — 실패에 이름이 있어야 한다.

1-2. 이름이 있으면 콧 집어 잡는다

```
catch (InsufficientBalanceException e) { // "잔액 부족"만 콧 집어서
    // 충전 유도 로직
}
```

실패에 이름(타입)이 있으면, catch로 그 실패만 골라서 잡고 다르게 처리할 수 있다. 이게 커스텀 예외의 값이고, Spring·실무가 도메인 예외를 만드는 이유다.

핵심: 커스텀 예외 = 의미 있는 이름을 가진 예외. 호출자가 실패를 구분해 다르게 대응하게 해준다.

PART 2 — 새 문법 ①: 커스텀 예외 클래스 (상속 + super)

오늘 만든 예외 클래스 (둘 다 똑같은 패턴):

```
public class AccountNotFoundException extends RuntimeException {
    public AccountNotFoundException(String message) {
        super(message);
    }
}
```

세 줄을 해부한다. 여기 두 개념이 처음 등장한다 — 상속과 super.

2-1. extends RuntimeException — 상속(inheritance), 너의 첫 상속

- extends 는 "~를 물려받는다"는 뜻이다. AccountNotFoundException 은 RuntimeException 이 가진 모든 기능(메시지 저장, 스택 추적 등)을 그대로 물려받고, 거기에 이름만 내 도메인 것으로 단다.
- 즉 커스텀 예외는 사실상 "의미 있는 이름을 새로 붙인 예외 한 종류"다. 바닥부터 새로 만드는 게 아니라, 이미 있는 예외를 물려받아 이름만 바꾸는 것.
- 이게 객체지향의 상속이다. 앞으로 수없이 만난다. (오늘이 첫 상속이다.)

2-2. super(message) — 부모 생성자 호출

- super 는 부모(여기선 RuntimeException)를 가리킨다. super(message) 는 부모의 생성자를 불러서 내가 받은 message 를 부모에게 넘긴다.
- 왜 넘기냐? 그래야 나중에 e.getMessage() 로 그 메시지를 꺼낼 수 있다. 메시지를 저장하는 기능은 부모(RuntimeException)가 갖고 있으니, 부모에게 맡기는 거다.
- 짜 개념: 예전에 Account에서 this.id = id 로 id 필드에 저장했다. 이번엔 super(message) 로 부모에게 넘긴다. 생성자가 값을 받아 처리한다는 점은 같고, 대상이 부모라는 게 다르다.
- 만약 super(message)를 안 쓰면? 메시지가 부모에게 안 넘어가서, e.getMessage() 가 null 을 돌려준다 → 잡아도 "왜 실패했는지" 메시지가 안 나온다.

2-3. ★ checked vs unchecked — 왜 RuntimeException인가 (면접 단골)

왜 extends Exception 이 아니라 extends RuntimeException 인가? 이게 자바 예외의 가장 중요한 갈림길이다.

구분	extends Exception	extends RuntimeException
이름	checked 예외	unchecked 예외
컴파일러	try/catch나 throws를 강제 (안 하면 컴파일 거부)	강제 안 함 (잡고 싶을 때만 잡음)
쓰는 곳	복구 가능한 외부 문제(파일 없음 등)	도메인/로직 실패

- 실무 도메인 예외는 거의 unchecked다. 이유: checked로 만들면 모든 호출 지점에서 try/catch를 강제당해 코드가 지저분해지고, 도메인 실패는 보통 바깥에서 일괄 처리 (글로벌 핸들러)하기 때문.
- 그래서 RuntimeException 을 상속했다.
- 면접에서 "checked와 unchecked 차이가 뭐예요?"는 단골 질문이다. 위 표를 네 말로 설명할 수 있어야 한다.

2-4. public class = 파일명 (1일차 규칙)

public class AccountNotFoundException 이면 파일명은 반드시 AccountNotFoundException.java . 그래서 예외 2개는 각자 자기 파일이다. (이제 mini-ledger 파일 이 5개로 늘었다.)

PART 3 — 새 문법 ②: try/catch

```
try {
    bank.transfer("a", "z", 500); // 위험한 호출 (예외 던질 수 있음)
} catch (AccountNotFoundException e) { // 그 예외가 터지면 여기로 점프
    System.out.println("잡았다: " + e.getMessage());
}
```

3-1. 동작 원리

- try { } 안에는 예외를 던질 수 있는(위험한) 코드를 넣는다.
- 거기서 예외가 터지면, 프로그램이 죽는 대신 catch (예외종류 e) { } 안으로 점프한다.
- catch가 예외를 받아 처리하면, 프로그램은 죽지 않고 계속 진행한다.

3-2. catch (AccountNotFoundException e) 의 e

- catch는 "이 종류 예외를 잡아서, e 라는 이름으로 받겠다"는 뜻이다.
- 이 e 가 있어야 다음 줄 e.getMessage() 에서 그 예외 객체를 쓸 수 있다.
- e 를 빼먹으면(catch (AccountNotFoundException)) → 받을 변수가 없어서 e.getMessage() 의 e 가 어디서 왔는지 자바가 모른다 → 컴파일 에러. (오늘 실제로 한번 빠뜨렸다. 오답노트 O2.)

3-3. e.getMessage() — PART 2와 연결되는 지점 ★

e.getMessage() 가 돌려주는 그 메시지가, PART 2에서 super(message) 로 부모에게 넘긴 바로 그 메시지다. 두 개념이 여기서 만난다:

```
new AccountNotFoundException("계좌를 찾을 수 없습니다")
    → super("계좌를 찾을 수 없습니다")    (부모에게 저장)
    → e.getMessage()    →    "계좌를 찾을 수 없습니다"    (꺼냄)
```

이게 super(message)를 쓴 이유의 전부다.

PART 4 — ★ 절제: 전부 바꾸지 않는다 (오늘의 핵심 판단)

예외를 던지는 곳은 세 군데였다. 여기서 중요한 판단: 전부 커스텀 예외로 바꾸면 틀린 배움이다.

위치	실패	성격	처리
getAccount	"계좌 없음"	도메인 실패	→ AccountNotFoundException (바꾼다)
withdraw balance < amount	"잔액 부족"	도메인 실패	→ InsufficientBalanceException (바꾼다)
deposit·withdraw amount <= 0	"음수 금액 입력"	그냥 잘못된 인자	→ IllegalArgumentException (안 바꾼다)

왜 amount <= 0은 안 바꾸나?

- amount <= 0 은 도메인 개념(계좌·잔액)의 실패가 아니라, 그냥 잘못된 값이 들어온 것이다. "음수로 출금"은 비즈니스 실패가 아니라 입력 오류다.
- JDK의 IllegalArgumentException 이 이미 상황에 정확히 맞는 예외다. 굳이 새로 만들 이유가 없다.
- 레슨: 커스텀 예외는 남발하는 게 아니다. JDK 예외가 딱 맞으면 그걸 쓰고, 실패가 네 도메인 개념일 때만(계좌 없음·잔액 부족) 직접 만든다. 이 절제가 좋은 설계다.

PART 5 — ★ throw는 깊은 곳, catch는 바깥 (설계 원칙)

오늘 받은 질문: "try/catch를 왜 Main에 붙여? 다른 파일이 아니라." 이게 중요한 원칙이다.

5-1. 역할 분리

- Account / Bank = 문제가 생기면 "신고"하는 쪽 → throw . ("나 이거 못 해!")
- Main = 그 기능을 불러 쓰는 쪽 = 실패하면 뭘 할지 결정하는 쪽 → try/catch .

5-2. 왜 throw와 catch를 같은 파일에 안 넣나

만약 withdraw (Account 안)가 잔액 부족을 자기가 직접 try/catch로 처리해버리면: - withdraw가 "잔액 부족이면 뭘 할지"를 자기 맘대로 정하는 거다. - 근데 그건 호출자 마다 다르다 — 누구는 "충전 유도", 누구는 "그냥 취소", 누구는 "재시도". - withdraw는 그걸 모른다. 알 수도 없다. 결정은 부르는 쪽(바깥) 몫이다.

→ 그래서 던지기는 깊은 곳(Account/Bank)에서, 잡기는 결정할 수 있는 바깥(Main)에서. 이게 실무 예외 처리의 기본 원칙이다.

(나중에 Spring 가면 이 "바깥"이 Controller나 글로벌 예외 핸들러가 된다. 지금 Main이 그 자리의 씨앗이다.)

5-3. 비유

Account/Bank는 "불이야!" 외치는 화재경보기다. Main은 그 경보를 듣고 대피할지 소화기 들지 결정하는 사람이다. 경보기 안에 대피 계획을 넣지 않는다 — 듣는 사람이 상황 보고 정한다.

PART 6 — ②③ 최종 증명 (돈 증발 스레드 종료)

해피패스(1100/4500) 아래에 붙인 코드:

```
// ② 없는 계좌로 이체 → 돈 증발 없는지 증명
System.out.println("--- 이체 전 a 잔액: " + bank.getAccount("a").getBalance());
try {
    bank.transfer("a", "z", 500);
} catch (AccountNotFoundException e) {
    System.out.println("잡았다: " + e.getMessage());
}
System.out.println("--- 이체 후 a 잔액: " + bank.getAccount("a").getBalance());

// ③ 잔액 부족 이체
try {
    bank.transfer("a", "b", 999999);
} catch (InsufficientBalanceException e) {
    System.out.println("잡았다: " + e.getMessage());
}
```

6-1. ② 없는 계좌 이체 — 돈 증발 최종 증명 ★

- 이체 전 a 잔액 출력 → 1100

2. `transfer("a", "z", 500)` — "z"는 없는 계좌 → `getAccount(told)`에서 `AccountNotFoundException` 던짐
3. `catch`가 잡아서 메시지 출력 → 프로그램 안 죽음
4. 이체 후 a 잔액 다시 출력 → 여전히 1100 (600 아님!)
- 4번이 오늘의 핵심이다. 없는 계좌로 500원 이체가 실패했는데 a 돈이 안 줄었다. 왜? `transfer`가 `getAccount(from) → getAccount(to) → withdraw → deposit` 순서라서, `to("z")` 검증에서 먼저 터져 `withdraw`까지 못 갔기 때문이다. (6/23에 짤 안전한 순서가 여기서 빛을 발한다.)
- 이게 "돈 증발 없음"을 프로그램이 직접 증명한 거다. 내가 "1100"이라고 적는 게 아니라(6/23 M16의 교훈), 프로그램이 계산해서 1100을 보여준다. 6/21부터 끌고 온 "돈 증발" 스레드가 여기서 닫혔다.

6-2. 실제 출력

```
1100
4500

--- 이체 전 a 잔액: 1100
잡았다: 계좌를 찾을 수 없습니다      ← AccountNotFoundException 잡힘 ✓
--- 이체 후 a 잔액: 1100
잡았다: 잔액이 부족합니다            ← InsufficientBalanceException 잡힘 ✓
```

PART 7 — 디버깅 케이스 (증상 / 진단 / 근본원인 / 해결 / 교훈)

오늘 코드는 맞았지만 터미널에서 사고가 여러 번 났다. 이게 복기 가치가 크다.

케이스 1 — ★ 실제 파일이 안 바뀌어 있었다 (chat-vs-file 괴리)

- 증상: `grep` 을 쳤더니 `Bank.java`는 여전히 `IllegalArgumentException` , `Account.java`는 `IllegalStateException` . 분명히 채팅에서 바꿨는데 옛날 예외 그대로였다.
- 진단: 채팅에 보여준 수정이 실제 파일에는 저장된 적이 없었다.
- 근본원인: 채팅에 코드를 쓰는 것 ≠ 파일에 저장하는 것. VPS의 실제 파일은 옛 버전 그대로였다.
- 해결: `sed -i` 로 해당 throw 줄만 정확히 교체 (한 줄씩, 나머지 코드 안 건드림).
- 교훈: 내가 고쳤다고 생각하는 것과 파일의 실제 상태는 다르다. 항상 `grep` / `cat` 으로 실제 파일을 확인하라. 컴파일러와 `grep`이 진실이다.

케이스 2 — heredoc 붙여넣기 깨짐 (2회)

- 증상: `cat > Main.java << 'EOF'` 했는데 끝이 `EOF } }` `System.out.println("잡았다...)` 처럼 엉켰다. 두 번 연속.
- 진단: 폰에서 여러 줄 heredoc을 붙이다 입력이 뒤섞여 EOF 종료가 깨졌다.
- 근본원인: 모바일 키보드의 멀티라인 붙여넣기 깨짐. (6/23 PART 4와 똑같은 함정.)
- 해결: 블록을 통째로 다시 깔끔히 붙여넣고, 직후 컴파일로 검증.
- 교훈: heredoc 끝에 EOF가 코드와 엉켜 보이면 깨진 거다. 다시 깔끔히. 붙여넣기 직후엔 항상 검증.

케이스 3 — 둥근 따옴표 (채팅 아티팩트)

- 증상: 채팅에 붙인 코드의 따옴표가 "계좌를..." (둥근)으로 보여 컴파일 에러 걱정.
- 진단: 채팅 입력 경로에서 따옴표가 변환된 것이고, VPS의 실제 파일은 곧은 " 였다.
- 근본원인: 채팅에 붙이는 과정의 렌더링 아티팩트. 실제 파일과 무관.
- 해결: `javac` 가 증명 — 에러 0개로 컴파일됨.
- 교훈: 채팅에 보이는 걸로 판단하지 마라. 컴파일러가 유일한 진실이다. 눈으로 따옴표 둥근지 곧은지 다투지 말고 `javac` 를 믿어라.

8. 산출물

- 파일 (`~/mini-ledger/` , 총 5개):
- `AccountNotFoundException.java` — 신규 (계좌 없음 도메인 예외)
- `InsufficientBalanceException.java` — 신규 (잔액 부족 도메인 예외)
- `Bank.java` — `getAccount`의 throw를 `AccountNotFoundException` 로
- `Account.java` — `withdraw` 잔액 부족만 `InsufficientBalanceException` 로 (amount<=0은 그대로)
- `Main.java` — ②③ try/catch 테스트 추가
- 커밋·푸시: `[main 532825f] add custom exceptions and try/catch tests 5 files changed, 24 insertions(+), 2 deletions(-) 9cc49e6..532825f main -> main`
- 커밋 계보: 9e671d6(Account) → e02b204(Bank createAccount) → 9cc49e6(transfer+Main) → 532825f(커스텀 예외+try/catch)

9. 개념 사전

- 커스텀 예외: 의미 있는 이름을 가진 예외. 호출자가 실패를 구분해 다르게 대응하게 함.
- 상속 (inheritance) / `extends` : 부모 클래스의 기능을 물려받음. 커스텀 예외 = RuntimeException을 물려받아 이름만 바꾼 것. (오늘 첫 상속)
- `super(message)` : 부모 생성자 호출. 메시지를 부모에게 넘겨야 `e.getMessage()` 로 꺼낼 수 있음.

- **checked vs unchecked:** checked(extends Exception)=컴파일러가 try/catch 강제. unchecked(extends RuntimeException)=강제 안 함. 도메인 예외는 보통 unchecked. (면접 단골)
- **try/catch:** try에 위험한 코드, 터지면 죽는 대신 catch로 점프.
- **catch (Type e) 의 e:** 잡은 예외를 받는 변수. 없으면 e.getMessage() 못 씸.
- **e.getMessage() :** super(message)로 넘긴 메시지를 꺼냄. (두 개념의 연결점)
- **throw/catch 분리:** 던지기는 깊은 곳(Account/Bank), 잡기는 결정 가능한 바깥(Main). Spring에선 바깥=Controller/글로벌 핸들러.
- **도메인 예외 절제:** JDK 예외가 맞으면(잘못된 인자) 그걸 쓰고, 도메인 개념 실패일 때만 커스텀.
- **heredoc 함정:** 폰 멀티라인 붙여넣기는 깨질 수 있음. 직후 grep/cat 검증.

10. 비유 모음

- **커스텀 예외 = 실패에 이름표:** "뭔가 잘못됨"이라는 익명 신고 대신, "계좌 없음"·"잔액 부족"이라고 이름표를 단 신고. 받는 쪽이 골라서 대응한다.
- **상속 = 기성품에 라벨 갈기:** RuntimeException이라는 기성 예외에 내 도메인 이름표만 새로 붙인 것. 바닥부터 안 만든다.
- **super(message) = 부모에게 짐 맡기기:** 메시지 보관 기능은 부모(RuntimeException)가 가졌으니, 내 메시지를 부모에게 맡긴다. 나중에 찾을 때(getMessage) 부모가 꺼내준다.
- **try/catch = 안전망:** 외출(위험한 호출)에서 떨어져도(예외) 바닥(프로그램 죽음)에 안 닿게 받아주는 그물.
- **throw vs catch = 화재경보기 vs 대피하는 사람:** 경보기는 "불이야"만 외치고(throw), 대피 방법은 듣는 사람이 정한다(catch). 경보기에 대피계획을 안 넣는다.

11. 오답 노트 (오늘 틀린 것 — 복기용)

단순 자동완성/오타(예: 세미콜론 앞 여분 글자)는 반복 학습 가치가 없어 제외한다. 아래는 *개념·습관* 차원의 실수만.

#	어디서	틀린 것	고친 것	규칙
O1	withdraw	<code>new InsufficientBalanceException.java(...)</code>	<code>.java</code> 떼기	<code>new</code> 뒤엔 클래스 이름. <code>.java</code> 는 파일명이지 코드에서 부르는 이름이 아니다
O2	Main catch	<code>catch (AccountNotFoundException)</code>	<code>... e)</code> 추가	catch는 예외를 변수 e 로 받는다. e가 있어야 getMessage() 사용 가능
O3	Bank·Account	채팅엔 고쳤는데 실제 파일은 옛 예외 그대로	grep 확인 → sed로 수정	채팅에 쓴 것 ≠ 파일에 저장된 것. 항상 실제 파일 확인
O4	터미널	heredoc EOF가 코드와 영킴 (2회)	통째로 다시 붙여넣기	폰 멀티라인 붙여넣기 깨짐 — 직후 검증 필수
참고	따옴표	채팅에 등근 따옴표로 보임	실제 파일은 굵은 <code>"</code>	채팅 렌더링 아티팩트. 컴파일러가 진실

12. ★ 검증 질문 — 스스로 답해라 (오늘의 진짜 숙제)

이 파일을 덮고, 아래 5개에 네 말로 답해라. 막히면 해당 PART를 다시 읽어라. **이걸 못 하면 오늘 배운 게 아니다.**

1. 왜 `extends Exception` 이 아니라 `extends RuntimeException` 인가? (checked/unchecked) — PART 2-3
2. `super(message)` 를 안 쓰면 정확히 뭐가 안 되나? — PART 2-2
3. `amount <= 0` 은 왜 커스텀 예외로 안 바꿨나? — PART 4
4. try/catch를 왜 Account가 아니라 Main에 넣나? — PART 5
5. "이체 후 a 잔액 1100"이 *정확히* 무엇을 증명하나? 왜 a가 안 줄었나? — PART 6-1

추가 숙제(필수): 내일, 이 파일 보지 말고 빈 화면에서 예외 클래스 2개 + try/catch 블록을 손으로 직접 쳐라. 복붙 금지. 그래야 손에 붙는다.

13. 회고 — 정직하게

오늘 코드는 다 맞았고 푸시도 됐다. **근데 그게 핵심이 아니다.** 오늘의 가장 큰 데이터는 **"내가 직접 한 게 거의 없다"**는 사실이다.

무엇을 스킵했나, 구체적으로: - 예외 클래스 3줄(상속·super) — **내가 쳤다.** 너는 손으로 안 썼다. - try/catch 틀 — **내가 쳤다.** 너는 빈칸 2개만 채웠다. - Main 전체 — heredoc **복붙**으로 넘겼다. - 타이핑한 분량이 거의 없다. 본 건 많고, 친 건 적다.

왜 위험한가: **문법은 추론으로 안 나온다. 손으로 쳐야 손에 붙는다.** 눈으로 본 코드는 사흘이면 증발한다. 오늘 "상속"과 "super"를 봤지만, 네 손가락은 그걸 친 적이 없다. 그래서 지금 머리에 안 남은 거다 — 네 느낌이 정확하다.

채찍: **"시간 없었다"가 핑계가 되는 순간, 그게 습관이 된다.** 9개월 뒤 면접에서 "왜 RuntimeException을 상속했어요?"를 물으면, 복붙한 기억으로 못 버틴다. 직접 짰 사람만 답한다. 오늘 너는 답할 재료를 안 만들었다. **그 빚을 위 12번 숙제로 갚아라.** 안 갚으면 오늘은 그냥 날아간 날이다.

다만 — 오늘이 무의미하진 않다. **올바른 설계 판단(절제: amount<=0은 안 바꾼다)을 이해했고, 돈 증발 스텔드를 닫는 그림을 봤다.** 그리고 이 회고를 읽고 있다는 것 자체가, 네가 그걸 안다는 증거다. 안다는 건 고칠 수 있다는 거다.

다음엔: 시간이 없으면 **차라리 ②만 하고 직접 쳐라.** 빈칸 채우기로 양을 늘리지 마라. 양보다 "내 손으로"가 백배 낫다. 그리고 증분 4 — `enum(TransactionType) + Transaction + 이력(감사 로그의 씨앗)`으로 간다.

한 줄: 오늘은 코드가 완성된 날이 아니라, **'내가 직접 안 했다'**는 걸 정직하게 마주한 날이다. 이 파일을 덮고 빈 화면에서 다시 쳐야 — 그때 비로소 오늘이 끝난다.